Project Viva + Technical Interview Preparation Report

Project Name: University Assignment Portal

This report is custom-tailored to your actual project architecture, codebase, database schemas, and workflows. It translates complex technical operations into simple, easy-to-explain answers suitable for an academic viva or a technical interview.

---

1. PROJECT OVERVIEW

What is the project?
The University Assignment Portal is a web application designed to help universities manage student assignment submissions digitally.

What problem does it solve?
It solves the problem of professors getting overwhelmed by too many assignment files on their email or WhatsApp. It centralizes all submissions, making them easy to track, grade, and organize.

Why was it created?
To keep academic submissions organized in one place, providing dedicated portals for students, professors, and administrators.

Who are the users?

1. **Admins**: Set up departments and create student/professor accounts.
2. **Students**: Select their professor and upload their PDF assignment files.
3. **Professors**: See submitted assignments, review the PDFs directly on the portal, and approve or reject them.

What is the complete workflow?
Admin Creates Account → System Emails Password → User Logs In → Forced Password Change → Student Uploads PDF → PDF Saved to Cloudinary → Professor Reviews PDF → Professor Approves/Rejects.

What technologies are used?

- **Frontend**: React, Vite, React Router, TailwindCSS/UI, Axios, Socket.io-client.
- **Backend**: Node.js, Express, Socket.io.
- **Database**: MongoDB with Mongoose.
- **Authentication**: JWT (JSON Web Tokens) with HttpOnly cookies, BcryptJS.
- **File Storage**: Cloudinary (via Multer).
- **Email**: Nodemailer.

Explain the complete architecture in a simple way:
My project uses a Client-Server architecture. The **Frontend** (React) runs in the user's browser. When a user performs an action, the frontend sends an HTTP request using Axios to the **Backend API** (Node.js/Express). The backend processes this request, communicates with the **Database** (MongoDB) to save or fetch data, or talks to **Cloudinary** for file uploads. The backend then sends a JSON response back to the frontend, which updates what the user sees on their screen.

Workflow: User → React Frontend → Axios (API Call) → Express Backend → MongoDB/Cloudinary → JSON Response → React Frontend Updates.

---

2. TECHNOLOGY STACK

React & Vite

- **Where used:** Frontend architecture.
- **Why used:** React makes it easy to build interactive user interfaces using reusable components. Vite is a build tool that makes React load and update incredibly fast during development.
- **Simple viva answer:** I used React to build a fast, single-page application (SPA) where pages load without refreshing the browser. Vite was used to make the development process much faster.

Node.js & Express

- **Where used:** Backend server.
- **Why used:** Node.js allows me to write backend logic using JavaScript. Express is a framework that makes it very easy to create API routes and handle HTTP requests and responses.
- **Simple viva answer:** I used Node.js and Express to create my backend API. Express handles the routes and connects my React frontend to my MongoDB database.

MongoDB & Mongoose

- **Where used:** Database.
- **Why used:** MongoDB is a NoSQL database that stores data in JSON-like documents, which perfectly matches JavaScript. Mongoose is used to create schemas (rules) for my data so it stays organized.
- **Simple viva answer:** I used MongoDB to store user data and assignment records because it's flexible and works seamlessly with Node.js.

JWT (JSON Web Tokens)

- **Where used:** Authentication system.
- **Why used:** To keep users logged in securely without storing session data on the server (stateless).
- **Simple viva answer:** I used JWT to securely identify users. After login, the backend gives a token, which the frontend stores in a secure cookie to prove the user's identity on every request.

Cloudinary

- **Where used:** File and Image management.
- **Why used:** Storing large PDF files or images directly in MongoDB is inefficient and slows down the database.
- **Simple viva answer:** I used Cloudinary to store the actual PDF assignment files uploaded by students, keeping my database lightweight. The database only stores the URL to the file.

Nodemailer

- **Where used:** Backend utility.
- **Why used:** To send emails from the server.
- **Simple viva answer:** I used Nodemailer to automatically send welcome emails with auto-generated passwords when an admin creates a new user, and to send OTPs for password resets.

Socket.io

- **Where used:** Frontend and Backend.
- **Why used:** For real-time updates without refreshing the page.
- **Simple viva answer:** I used Socket.io to implement real-time chat and show who is currently online in the portal.

---

3. JWT & AUTHENTICATION WORKFLOW

Q: How does JWT authentication work in your project?
A: When a user logs in, the backend checks the email and password. If correct, the backend generates a JWT token containing the user's ID and role. Instead of sending it back in the JSON body, my backend automatically saves the token inside a secure, `HttpOnly` cookie in the user's browser. When the frontend makes future API calls (using Axios with `withCredentials: true`), the browser automatically attaches this cookie. The backend middleware (`verifyStudent`, `verifyProfessor`, etc.) then reads the cookie, verifies the token's secret key, and allows access to protected routes.

Q: Where is the token stored and why?
A: The token is stored in an `HttpOnly` cookie. This is much more secure than `localStorage` because JavaScript (and potentially malicious XSS attacks) cannot read `HttpOnly` cookies.

Q: How are passwords stored?
A: Passwords are never stored as plain text. Before saving a user in MongoDB, my Mongoose schema uses a `pre("save")` hook to hash the password using `bcryptjs` with a salt factor of 10. During login, `bcrypt.compare()` checks if the entered password matches the stored hash.

Q: How does the auto-generated password system work?
A: When an Admin creates a new user, if they don't provide a password, my backend generates an 8-character random string using `Math.random()`. The user is saved with a flag `mustChangePassword: true`. When the user logs in for the first time, the server sees this flag and tells the React frontend to redirect them to a "Force Change Password" page, locking them out of the dashboard until they set their own private password.

Q: How is logout handled?
A: The frontend calls the `POST /logout` route. The backend responds by clearing the `HttpOnly` cookie containing the JWT, completely destroying the user's session.

---

4. DATABASE AND SCHEMA

Q: Why did you choose MongoDB?
A: MongoDB stores data as documents, which looks exactly like JSON. Since my frontend (React) and backend (Node.js) both use JavaScript/JSON natively, it made transferring and saving data very natural and fast.

Q: How did you design the database schemas?
A: I designed models based on the entities in my system.

1. **UserData**: Stores Students, Professors, and HODs. It contains fields like `role` to distinguish them, and an ObjectId reference to their `Department`.
2. **Admin**: A separate model just for administrators for added security.
3. **Assignment**: Stores submission details. It has fields referencing the Student (ObjectId), the Professor (ObjectId), the Cloudinary file URL, and the `status` (Draft, Submitted, Approved, Rejected).

Q: Why did you use ObjectId references?
A: Instead of duplicating a professor's name and email inside every single assignment they receive, I store their unique ObjectId. This keeps the database normalized. When I need the professor's details, I use Mongoose's `.populate()` to fetch the related data automatically.

---

5. ROUTES AND API WORKFLOW

Q: How does a request move through the backend?
A:

1. The frontend Axios client sends an HTTP request to an endpoint (e.g., `POST /api/student/assignments/upload`).
2. It first hits the **Router** (`studentRoutes.js`).
3. The router passes it through **Authentication Middleware** which reads the JWT cookie to ensure a student is logged in.
4. Next, it passes through **Multer Middleware** which temporarily saves the uploaded PDF to the server's disk.
5. Finally, it reaches the **Controller** (`assignmentController.js`), which uploads the PDF to Cloudinary, deletes the temporary file, saves the file URL to MongoDB, and returns a JSON success response to the frontend.

Q: Why did you use GET, POST, PUT, and DELETE?
A:

- **GET**: Used to retrieve data (e.g., getting the list of professors or stats for the dashboard).
- **POST**: Used to create new data or send secure data (e.g., submitting an assignment, logging in).
- **PUT/PATCH**: Used to update existing data (e.g., professor updating the status of an assignment to "Approved").
- **DELETE**: Used to remove data from the database.

---

6. CLOUDINARY & FILE UPLOAD (Crucial)

Q: How does PDF upload work in your project?
A: It is a multi-step process for efficiency and security:

1. The student selects a PDF (`< 10MB`).
2. The React frontend sends it using `FormData` via Axios to the backend.
3. The backend uses `multer` to temporarily save the file to the server's local temporary folder (`os.tmpdir()`).
4. The backend then immediately calls `cloudinary.uploader.upload()` to send that temporary file to the Cloudinary cloud storage under the `raw` resource type

(since it's a PDF document, not an image).
5. Cloudinary returns a secure URL for the file.
6. The backend saves this URL in the MongoDB `Assignment` document.
7. **Crucial Step:** The backend runs `fs.unlinkSync()` to delete the temporary file from the local server disk so the server doesn't run out of storage space.

Q: How does the Professor view the file?
A: The React frontend receives the Cloudinary URL from the database and embeds it directly into an `<iframe>`, allowing the professor to read the PDF right on the webpage without downloading it.

---

7. EMAIL (Nodemailer)

Q: How does your system send emails?
A: I used the `Nodemailer` library. My backend uses SMTP credentials (like `SMTP_USER` and `SMTP_PASS`) stored safely in my `.env` file.
When an admin creates a new user, the backend generates an HTML template containing the user's login details and auto-generated password. Nodemailer then connects to the email server and dispatches it automatically. I also use this for sending a 4-digit OTP during the "Forgot Password" flow, storing the OTP in the database with a 15-minute expiry (`resetPasswordExpires`).

---

8. FRONTEND (React)

Q: How do you handle State in React?
A: I primarily use React Hooks like `useState` for local component state (like form inputs) and `useEffect` for fetching data from the API as soon as a component mounts.

Q: How are protected pages handled in React?
A: I use Layout components (like `StudentLayout`, `AdminLayout`). When these load, they ping the `/api/auth/me` endpoint. If the backend rejects it (invalid JWT), the user is redirected to the login page. I do not store the JWT in `localStorage`, only basic non-sensitive profile info, because the backend reads the secure cookie directly.

Q: How are errors displayed to the user?
A: I use `try/catch` blocks around my Axios calls. If an error occurs, I catch it and use `react-hot-toast` to show a beautiful notification popup reading the error message sent directly from my backend controller (e.g., `toast.error(err.response.data.message)`).

---

9. SECURITY QUESTIONS

Q: How is your project secured?
A:

1. **Passwords**: Hashed using `BcryptJS`.
2. **Authentication**: Uses stateless JWT tokens.
3. **Token Storage**: Stored in `HttpOnly, Secure` cookies to prevent XSS (Cross-Site Scripting) token theft.
4. **Environment Variables**: Sensitive keys (MongoDB URI, Cloudinary Secret, JWT Secret) are hidden inside a `.env` file and never uploaded to GitHub.
5. **File Validation**: `Multer` strictly enforces that only `application/pdf` files under 10MB can be uploaded, preventing malicious script uploads.

---

10. ERROR HANDLING & DEPLOYMENT

Q: What happens if an API request fails?
A: The backend has a global error handler in `server.js`. If anything crashes, it catches the error and sends a clean `500 Server Error` JSON response instead of shutting down the node process. The React frontend catches this 500 status and displays a toast notification to the user.

Q: How did you configure environment variables for deployment?
A: Locally, they are in a `.env` file. In production (like on Vercel), I manually entered the environment variables into the Vercel dashboard settings so the deployed app can securely access the production database and Cloudinary account. The Axios client is set up to automatically switch between `localhost` and the deployed URL based on the environment.

---

11. TRICKY PROFESSOR FOLLOW-UP QUESTIONS

Professor: "Why did you use JWT instead of standard sessions?"
Student: "Because JWT helps authenticate users without storing session data on the server."
Professor: "Okay, but if the JWT is stored on the client side, how do you prevent hackers from stealing it using JavaScript (XSS)?"
Ideal Answer: "In my project, the JWT is NOT stored in `localStorage` where JavaScript can read it. My backend sends the token back as an `HttpOnly` cookie. This tells the browser to lock the cookie away from JavaScript entirely. The browser just automatically attaches it to subsequent API requests."

Professor: "Why did you use Cloudinary? Why not just save the PDF directly into MongoDB?"
Ideal Answer: "MongoDB has a strict 16MB document size limit. If I saved PDFs as binary data in MongoDB, it would bloat the database, cause slow search queries, and quickly hit that limit. Cloudinary handles large files efficiently, acts as a CDN for fast loading, and leaves my database lightweight by only storing the URL string."

---

12. TOP 30 MOST IMPORTANT VIVA QUESTIONS (To Memorize)

Very High Priority

1. **What is the complete architecture of your project?** (React Frontend -> Express Backend -> MongoDB/Cloudinary)
2. **How does JWT work in YOUR code?** (Generated on login, stored in HttpOnly cookie, verified by authMiddleware on protected routes).
3. **How exactly does a student upload a PDF?** (FormData -> Multer tmp folder -> Cloudinary upload -> fs.unlinkSync to delete tmp -> MongoDB saves URL).
4. **How do you secure passwords?** (BcryptJS hashing in Mongoose pre-save hook).
5. **Why NoSQL/MongoDB instead of SQL?** (Document structure perfectly matches JavaScript JSON objects, providing flexibility for different user roles).
6. **How does the force-password-change work?** (Backend generates random password, sets `mustChangePassword: true`, frontend intercepts login and routes to change-password screen).
7. **How do you prevent XSS attacks?** (HttpOnly cookies for JWTs).
8. **What does Multer do?** (Parses multipart/form-data so the backend can read the uploaded physical file).
9. **How do you handle backend errors?** (Global error middleware catching exceptions and returning JSON instead of crashing the server).

10. **Explain your database schema for Assignments.** (Stores Title, Student ObjectId, Professor ObjectId, Cloudinary URL, and Status).

High Priority

11. **Why React?** (Component-based SPA for fast, seamless user experience without page reloads).
12. **What is middleware in Express?** (Functions that run between the request and the final controller, like my `authMiddleware` or `multer`).
13. **How does Nodemailer work here?** (Connects via SMTP env credentials to dispatch welcome emails and OTPs).
14. **How are roles handled?** (The `role` field in UserData schema; backend middleware checks if `req.user.role === 'admin'`).
15. **What is Axios?** (An HTTP client used in React to talk to the backend, configured with `withCredentials: true` for cookies).
16. **How does the Professor view the PDF?** (The Cloudinary URL is embedded into an `<iframe>` in the React component).
17. **What is the use of `populate()` in Mongoose?** (To replace ObjectId references with the actual user details from the other collection).
18. **Why Vite?** (Extremely fast build tool and dev server compared to Create React App).
19. **How do you handle routing in React?** (`react-router-dom` in `App.jsx`).
20. **What happens if a user uploads a 50MB file?** (Multer is configured to reject files over 10MB, throwing a `LIMIT_FILE_SIZE` error gracefully).

Medium Priority

21. **How does Socket.io work here?** (Real-time bidirectional communication for live chat and online status).
22. **What hooks did you use in React?** (`useState` for form data, `useEffect` for fetching data on mount).
23. **What is CORS?** (Cross-Origin Resource Sharing. Enabled in backend to allow requests from the React frontend running on a different port/domain).
24. **How do you store the secret keys?** (In a `.env` file, ignored by Git).
25. **What happens on Logout?** (Backend clears the HttpOnly cookie).
26. **What is a Mongoose Model?** (A wrapper around the schema that allows querying the database).
27. **How do you handle validation?** (Frontend HTML5 validation, backend checks for missing fields before processing).
28. **Why didn't you use Redux?** (The app's state was simple enough to manage with local state and context/prop drilling).
29. **What was the hardest part of the project?** (Explain a genuine struggle, likely the Cloudinary integration or setting up HttpOnly cookies across different domains).
30. **How did you deploy it?** (Frontend on Vercel, Backend on a Node hosting service, Database on MongoDB Atlas).

---

13. PROJECT EXPLANATIONS

2-Minute Explanation

"My project is a University Assignment Portal built using the MERN stack (MongoDB, Express, React, Node.js). It solves the problem of professors receiving disorganized assignments via email. The system has three main actors: Admins who manage accounts, Students who upload PDF assignments, and Professors who review and grade them online. I used React for a fast frontend, Express/Node.js for the backend API, MongoDB to store user and assignment metadata, and Cloudinary to securely store the actual PDF files. Security is prioritized by using Bcrypt to hash passwords and JWTs stored in HttpOnly cookies for secure authentication."

5-Minute Detailed Explanation

(Start with the 2-minute intro, then add the technical workflow)
"To walk you through a core technical workflow: When an Admin creates a new student, the backend auto-generates a secure password and uses Nodemailer to email it to the student. When the student logs in, the backend verifies the credentials using Bcrypt, and issues a JWT token hidden in an HttpOnly cookie to protect against cross-site scripting (XSS). The system detects the auto-generated password flag and forces the student to set a private password.

When the student submits an assignment, my React frontend sends the PDF via Axios using FormData. On the Express backend, Multer middleware intercepts this, temporarily saves the PDF to the server disk, checks that it is under 10MB, and then pushes it to Cloudinary. Cloudinary returns a secure URL. The backend immediately deletes the temporary local file to save server space, and saves the Cloudinary URL into the MongoDB Assignment document alongside the Student and Professor ObjectIds. Finally, when the Professor logs in, the React frontend fetches that Cloudinary URL and displays the PDF inside an iframe, allowing the professor to read and approve the assignment without ever needing to download it. This architecture ensures the database remains fast, the server disk remains empty, and the user experience remains seamless."